Programación 1 — Unidad 9: Manejo de archivos

Actividad 3: Procesamiento manual de archivos CSV

Índice

9. Manejo de archivos

– 9.9. Qué es un archivo CSV

– 9.10. Procesar texto: strip(), split() y join()

– 9.11. Leer un CSV sin librerías

– 9.12. El módulo csv: reader y writer

– 9.13. Trabajar con diccionarios: DictReader y DictWriter



9.9. Qué es un archivo CSV

Un archivo **CSV** (*Comma Separated Values* o Valores Separados por Comas) es un archivo de texto plano que guarda datos en forma de tabla. La mejor forma de entenderlo es compararlo con una planilla de cálculo como **Microsoft Excel** o **Google Sheets**: cada **línea** es una **fila**, cada valor separado por coma es una **celda**, y la primera línea suele contener los nombres de las columnas (la **cabecera**).

```
nombre,edad,pais
Ana,30,Argentina
Juan,25,Argentina
Lucia,41,Uruguay
```

Por esa estructura, cualquier planilla de cálculo abre un CSV y lo muestra en celdas. Y en desarrollo se usa muchísimo por tres razones: es **interoperable** (un sistema en Java exporta un CSV que un script en Python procesa sin problemas), es **liviano** (texto plano, sin

formatos ni fórmulas, ocupa mucho menos que un `.xlsx`) y sirve para **persistir** los resultados de un programa en el disco.

Nota

Aunque el nombre indica «comas», en muchos países (como Argentina) es común usar el **punto y coma (`;`)** como separador, ya que la coma se reserva para los decimales. Más adelante vamos a ver cómo adaptar el código a cualquiera de los dos casos con el parámetro `delimiter`.

9.10. Procesar texto: strip(), split() y join()

Antes de leer un CSV necesitamos tres métodos de texto. Con ellos vas a poder **limpiar** una línea, **separarla** en datos y volver a **unirla**. Son la base de todo el procesamiento manual.

Método	Qué hace	Ejemplo → resultado
`.strip()`	Quita espacios y saltos de los extremos	`" hola \n".strip()` → `"hola"`
`.split(sep)`	Divide el texto en una lista	`"a,b,c".split(",")` → `["a","b","c"]`
`sep.join(lista)`	Une una lista en un texto	`",".join(["a","b"])` → `"a,b"`

`strip()` elimina los **caracteres en blanco** al inicio y al final de una cadena: espacios, saltos de línea (`\n`) y tabulaciones (`\t`). Es imprescindible porque al leer una línea de un archivo, esa línea **siempre termina con un `\n`** que ensucia el último dato.

`split()` divide una cadena en una **lista de partes** usando un separador. En un CSV ese separador es la coma, y así accedemos a cada campo del registro por separado. En la práctica los dos se usan juntos:

```
linea = "  Juan,25,Argentina\n"
campos = linea.strip().split(",")
print(campos)

# ['Juan', '25', 'Argentina']
```

Qué hace el código:

- Primero corre `strip()`, que devuelve la línea sin el salto de línea ni los espacios de los extremos.
- Sobre **ese resultado** corre `split(",")`, que lo corta usando la coma.
- El orden importa: al revés, el último campo quedaría como `'Argentina\n'`.
- Todos los elementos son **strings**, incluso `'25'`. Si necesitás el número, convertilo con `int()`.

`.join()` es la operación inversa: pertenece a los **strings** y une los elementos de una lista en un solo texto, usando como **separador** el string que lo llama. Para armar una línea de CSV, el separador es la coma:

```
campos = ["Juan", "25", "Argentina"]
linea = ",".join(campos)
print(linea)

# Juan,25,Argentina
```

Importante

Todos los elementos del iterable **deben ser strings**. Si hay números (`int`, `float`, etc.), primero hay que convertirlos con `str()`, o Python lanza un `TypeError`. Por ejemplo: `",".join([str(x) for x in [1, 2, 3]])`.

9.11. Leer un CSV sin librerías

Con `strip()` y `split()` ya podés procesar un CSV completo usando solo lo que aprendiste en la Actividad 2. Este es el patrón:

```
with open("personas.csv", "r", encoding="utf-8") as archivo:
    encabezados = archivo.readline().strip().split(",")
    print(f"Columnas: {encabezados}")

    for linea in archivo:
        campos = linea.strip().split(",")
        nombre = campos[0]
        edad = int(campos[1])
        print(f"{nombre} tiene {edad} años")
```

Qué hace el código:

- `readline()` consume la **primera línea** (la cabecera) y deja el puntero en la segunda: por eso el `for` arranca directamente en los datos.
- `campos[0]` y `campos[1]` acceden por **posición**: hay que recordar el orden de las columnas.
- `int(campos[1])` convierte la edad, porque al leer de un archivo **todo viene como string**.

Y para **escribir** un CSV a mano, `join()` hace el trabajo inverso:

```
personas = [["nombre", "edad"], ["Ana", "30"], ["Juan", "25"]]

with open("personas.csv", "w", encoding="utf-8") as archivo:
    for fila in personas:
        archivo.write(",".join(fila) + "\n")
```

Importante

El procesamiento manual tiene un límite: **falla si un dato contiene una coma**. Si el país fuera `"Buenos Aires, Argentina"`, el `split(",")` lo partiría en dos campos y todo el registro se correría. Para eso existe el módulo `csv`. Aun así, **entender `strip()` y `split()` es esencial** para saber cómo funcionan las cosas «por debajo».

9.12. El módulo csv: reader y writer

Python incluye el módulo `csv` en su **biblioteca estándar**: ya viene instalado, solo hay que importarlo con `import csv`. Al abrir el archivo conviene sumar `encoding="utf-8"`, que es el «traductor» que le dice a Python cómo interpretar los bytes; sin él, un archivo con tildes o eñes puede fallar o mostrar símbolos extraños.

`csv.reader(archivo)` crea un **iterador**: no lee todo de golpe, procesa el archivo línea por línea a medida que lo recorrés, y convierte cada línea en una **lista de Python**. Como no distingue la cabecera de los datos, la salteamos con `next()`:

```
import csv

with open("alumnos.csv", "r", encoding="utf-8") as archivo:
    lector = csv.reader(archivo)

    # Extraemos la primera fila (encabezados)
    encabezados = next(lector)
    print(f"Columnas detectadas: {encabezados}")

    # Ahora el bucle empieza directamente en los datos
    for fila in lector:
        # fila es una lista, ej: ["Juan", "20", "8.5"]
        print(f"Alumno: {fila[0]} - Calificación: {fila[2]}")
```

Si el archivo usa punto y coma en lugar de comas, se lo indicás con `delimiter`: `csv.reader(archivo, delimiter=";")`.

Para **escribir**, `csv.writer` recibe listas y se encarga solo de las comas y los saltos de línea. Al abrir en modo `"w"` hay que sumar `newline=""`:

```
alumnos = [
    ["Nombre", "Materia", "Nota"],
    ["Matias", "Programación", "9"],
    ["Lucía", "Sistemas", "10"]
]

with open("notas.csv", "w", newline="", encoding="utf-8") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerows(alumnos)          # todas las filas de golpe
    # escritor.writerow(["Ana", "Bases", "8"])   # una sola fila
```

Importante

`newline=""` es el parámetro más importante en Windows: sin él, Python puede agregar filas vacías entre cada dato y corromper el formato del CSV. Y recordá que el modo `"w"` **borra todo el contenido previo** del archivo.

9.13. Trabajar con diccionarios: DictReader y DictWriter

A diferencia del `reader`, que devuelve listas, el **DictReader** convierte cada fila en un **diccionario**: usa la primera línea del archivo como **claves** y el contenido de cada celda como **valor**. No hace falta `next()`: consume la cabecera automáticamente.

```
# productos.csv →  id,nombre,precio
#                  1,Teclado,5000

with open("productos.csv", "r", encoding="utf-8") as archivo:
    lector_dict = csv.DictReader(archivo)

    for fila in lector_dict:
        # fila es {'id': '1', 'nombre': 'Teclado', 'precio': '5000'}
        print(f"Producto: {fila['nombre']} - Precio: ${fila['precio']}")
```

Característica	csv.reader	csv.DictReader
Formato de salida	Lista (`list`)	Diccionario (`dict`)
Acceso a datos	Por índice: `fila[0]`	Por nombre: `fila['nombre']`
Encabezados	Deben saltarse con `next()`	Se usan automáticamente como claves
Ideal para…	Archivos simples o sin títulos	Archivos estructurados y código limpio

Buenas prácticas

Legibilidad: es mucho más claro escribir `fila['precio']` que recordar que el precio estaba en `fila[2]`.

Mantenimiento: si mañana el CSV cambia el orden de las columnas, el código **sigue funcionando**, porque busca por nombre de clave y no por posición.

Para escribir hay un equivalente, **DictWriter**, que obliga a definir primero los nombres de las columnas. Es la forma más segura: si un diccionario trae una clave que no está en `fieldnames`, Python avisa con un error en lugar de guardar datos en columnas inexistentes.

```
empleados = [
    {"id": 101, "nombre": "Roberto", "puesto": "Desarrollador"},
    {"id": 102, "nombre": "Elena", "puesto": "Analista"}
]
columnas = ["id", "nombre", "puesto"]

with open("nomina.csv", "w", newline="", encoding="utf-8") as archivo:
    escritor_dict = csv.DictWriter(archivo, fieldnames=columnas)
    escritor_dict.writeheader()      # ¡la fila de títulos!
    escritor_dict.writerows(empleados)
```

Qué hace el código:

- `fieldnames=columnas` define el **orden** de las columnas: no importa en qué orden estén las claves dentro de tus diccionarios.
- `writeheader()` genera automáticamente la fila de títulos.
- Si te olvidás de `writeheader()`, el archivo queda sin cabecera y después `DictReader` va a tomar la primera fila de datos como si fueran los títulos.

Encontrá el error

Este programa tendría que imprimir la edad de cada persona, pero rompe en la primera vuelta. Fijate qué se está leyendo antes de los datos.

```
with open("personas.csv", "r", encoding="utf-8") as f:
    for linea in f:
        campos = linea.strip().split(",")
        print(int(campos[1]))
```

Qué estaba mal

La primera línea del archivo es la cabecera (nombre,edad,pais), así que campos[1] vale el texto "edad" y int("edad") lanza un ValueError. Hay que saltear la cabecera antes del bucle, con f.readline() si lo hacés a mano o con next(lector) si usás csv.reader.

Para reflexionar y autoevaluarte

— ¿En qué caso concreto elegirías `csv.reader` en lugar de `csv.DictReader`?

— ¿Qué problema aparece si un dato del CSV contiene una coma y lo procesás con `split(",")`?

Antes de seguir, verificá que podés:

— Limpiar una línea con `strip()`, separarla con `split()` y rearmarla con `join()`.

— Leer un CSV a mano, salteando la cabecera.

— Leer y escribir CSV con `csv.reader` y `csv.writer`.

— Explicar la diferencia entre `reader` y `DictReader`, y cuándo conviene cada uno.

En la **Actividad 4** vas a aprender a que todo esto no se rompa: qué hacer cuando el archivo no existe, cuando el dato no se puede convertir o cuando pasa algo que no habías previsto.